

Tableaux JavaScript

Dans ce chapitre, vous apprendrez à créer et à manipuler des tableaux en JavaScript.

Qu'est-ce qu'un tableau

Les tableaux sont des variables complexes qui nous permettent de stocker plus d'une valeur ou un groupe de valeurs sous un seul nom de variable. Les tableaux JavaScript peuvent stocker n'importe quelle valeur valide, y compris des chaînes, des nombres, des objets, des fonctions et même d'autres tableaux, permettant ainsi de créer des structures de données plus complexes telles qu'un tableau d'objets ou un tableau de tableaux.

Supposons que vous souhaitiez stocker le nom des couleurs dans votre code JavaScript. Stocker les noms de couleurs un par un dans une variable pourrait ressembler à ceci

```
let color1 = "Red";  
let color2 = "Green";  
let color3 = "Blue";
```

Mais que se passe-t-il si vous avez besoin de stocker les noms d'état ou de ville d'un pays dans des variables et cette fois ce n'est pas seulement trois peut être cent. Il est assez difficile et ennuyeux de stocker chacun d'eux dans une variable distincte. De plus, utiliser autant de variables simultanément et en garder une trace sera une tâche très difficile. Et ici, le tableau entre en jeu. Les tableaux résolvent ce problème en fournissant une structure ordonnée pour stocker plusieurs valeurs ou un groupe de valeurs.

Création d'un tableau

Le moyen le plus simple de créer un tableau en JavaScript consiste à placer une liste de valeurs séparées par des virgules entre crochets (`[]`), comme indiqué dans la syntaxe suivante :

```
let monTableau = [ élément0 , élément1 , ..., élémentN ];
```

Un tableau peut également être créé à l'aide du `Array()` constructeur, comme indiqué dans la syntaxe suivante. Cependant, pour des raisons de simplicité, la syntaxe précédente est recommandée.

```
let monTableau = new Array( élément0 , élément1 , ..., élémentN );
```

Voici quelques exemples de tableaux créés à l'aide de la syntaxe littérale de tableau :

```
const colors = ["Red", "Green", "Blue"];  
const fruits = ["Apple", "Banana", "Mango", "Orange", "Papaya"];  
const cities = ["London", "Paris", "New York"];  
const person = ["John", "Wick", 32];
```

Remarque : Un tableau est une collection ordonnée de valeurs. Chaque valeur d'un tableau est appelée un élément, et chaque élément a une position numérique dans un tableau, appelée son index.

Accéder aux éléments d'un tableau

Les éléments du tableau sont accessibles par leur index en utilisant la notation entre crochets. Un index est un nombre qui représente la position d'un élément dans un tableau.

Les index de tableau sont basés sur zéro. Cela signifie que le premier élément d'un tableau est stocké à l'index 0, et non 1, le deuxième élément est stocké à l'index 1, et ainsi de suite. Les index de tableau commencent à 0 et augmentent jusqu'au nombre d'éléments moins 1. Ainsi, un tableau de cinq éléments aurait des index de 0 à 4.

L'exemple suivant vous montrera comment obtenir des éléments de tableau individuels par leur index.

```
const fruits = ["Apple", "Banana", "Mango", "Orange", "Papaya"];

document.write(fruits[0] + "<br>"); // Prints: Apple
document.write(fruits[1] + "<br>"); // Prints: Banana
document.write(fruits[2] + "<br>"); // Prints: Mango
document.write(fruits[fruits.length - 1]); // Prints: Papaya
```

Remarque : En JavaScript, les tableaux ne sont en fait qu'un type spécial d'objets qui ont des index numériques comme clés. L'opérateur `typeof` renverra "object" pour les tableaux.

Obtenir la longueur d'un tableau

La propriété **length** renvoie la longueur d'un tableau, qui est le nombre total d'éléments contenus dans le tableau. La longueur du tableau est toujours supérieure à l'indice de l'un de ses éléments.

```
const fruits = ["Apple", "Banana", "Mango", "Orange", "Papaya"];  
document.write(fruits.length); // Prints: 5
```

Boucle à travers les éléments du tableau

Vous pouvez utiliser la boucle **for** pour accéder à chaque élément d'un tableau dans un ordre séquentiel, comme ceci

```
for(let i = 0; i < fruits.length; i++){  
    document.write(fruits[i] + "<br>"); // Print array element  
}
```

ECMAScript 6 a introduit un moyen plus simple d'itérer sur un élément de tableau, qui est une boucle **for-of**. Dans cette boucle, vous n'avez pas besoin d'initialiser et de suivre la variable du compteur de boucle (i).

Voici le même exemple réécrit en utilisant la boucle **for-of**:

```
for(let fruit of fruits){  
    document.write(fruit + "<br>"); // Print array element  
}
```

Vous pouvez également parcourir les éléments du tableau à l'aide de la boucle **for-in**, comme ceci :

```
for(let i in fruits) {  
    document.write(fruits[i] + "<br>");  
}
```

Ajout de nouveaux éléments à un tableau

Pour ajouter un nouvel élément à la fin d'un tableau, utilisez simplement la méthode `push()`, comme ceci :

```
let colors = ["Red", "Green", "Blue"];  
colors.push("Yellow");
```

De même, pour ajouter un nouvel élément au début d'un tableau, utilisez la méthode `unshift()`, comme ceci :

```
let colors = ["Red", "Green", "Blue"];  
colors.unshift("Yellow");
```

Vous pouvez également ajouter plusieurs éléments à la fois en utilisant les méthodes `push()` et `unshift()` , comme ceci :

```
let colors = ["Red", "Green", "Blue"];  
colors.push("Pink", "Violet");  
colors.unshift("Yellow", "Grey");
```

Supprimer des éléments d'un tableau

Pour supprimer le dernier élément d'un tableau, vous pouvez utiliser la méthode `pop()`. Cette méthode renvoie la valeur qui a été extraite. Voici un exemple

```
let colors = ["Red", "Green", "Blue"];  
let last = colors.pop();  
  
document.write(last + "<br>"); // Prints: Blue  
document.write(colors.length); // Prints: 2
```

De même, vous pouvez supprimer le premier élément d'un tableau à l'aide de la méthode `shift()`. Cette méthode renvoie également la valeur qui a été décalée. Voici un exemple

```
let colors = ["Red", "Green", "Blue"];  
let first = colors.shift();  
  
document.write(first + "<br>"); // Prints: Red  
document.write(colors.length); // Prints: 2
```


Ajout ou suppression d'éléments à n'importe quelle position

La méthode `splice()` est une méthode de tableau très polyvalente qui vous permet d'ajouter ou de supprimer des éléments de n'importe quel index, en utilisant la syntaxe `array.splice(startIndex, deleteCount, elem1, ..., elemN)`.

Cette méthode prend trois paramètres : le premier paramètre est l'indice auquel commencer à épisser le tableau, il est obligatoire ; le deuxième paramètre est le nombre d'éléments à supprimer (à utiliser 0 si vous ne souhaitez supprimer aucun élément), il est facultatif ; et le troisième paramètre est un ensemble d'éléments de remplacement, il est également facultatif. L'exemple dans le slide suivant montre comment cela fonctionne :

```
let colors = ["Red", "Green", "Blue"];
let removed = colors.splice(0,1); // supprime le premier element

document.write(colors + "<br>"); // Prints: Green,Blue
document.write(removed + "<br>"); // Prints: Red (tableau à un élément)
document.write(removed.length + "<br>"); // Prints: 1

removed = colors.splice(1, 0, "Pink", "Yellow"); // Insérer deux éléments à la position un
document.write(colors + "<br>"); // Prints: Green,Pink,Yellow,Blue
document.write(removed + "<br>"); // Empty array
document.write(removed.length + "<br>"); // Prints: 0

removed = colors.splice(1, 1, "Purple", "Voilet"); // Insérer deux valeurs, en supprimer une
document.write(colors + "<br>"); //Prints: Green,Purple,Voilet,Yellow,Blue
document.write(removed + "<br>"); // Prints: Pink (tableau à un élément)
document.write(removed.length); // Prints: 1
```

La méthode `splice()` renvoie un tableau des éléments supprimés, ou un tableau vide si aucun élément n'a été supprimé, comme vous pouvez le voir dans l'exemple ci-dessus. Si le deuxième argument est omis, tous les éléments du début à la fin du tableau sont supprimés. Contrairement aux méthodes `slice()` et `concat()`, la méthode `splice()` modifie le tableau sur lequel elle est appelée.

Création d'une chaîne à partir d'un tableau

Il peut arriver que vous vouliez simplement créer une chaîne en joignant les éléments d'un tableau. Pour ce faire, vous pouvez utiliser la méthode `join()`. Cette méthode prend un paramètre facultatif qui est une chaîne de séparation ajoutée entre chaque élément. Si vous omettez le séparateur, JavaScript utilisera la virgule (,) par défaut. L'exemple suivant montre comment cela fonctionne :

```
let colors = ["Red", "Green", "Blue"];

document.write(colors.join() + "<br>"); // Prints: Red,Green,Blue
document.write(colors.join("") + "<br>"); // Prints: RedGreenBlue
document.write(colors.join("-") + "<br>"); // Prints: Red-Green-Blue
document.write(colors.join(", ")); // Prints: Red, Green, Blue
```

Vous pouvez également convertir un tableau en une chaîne séparée par des virgules à l'aide de `toString()`. Cette méthode n'accepte pas le paramètre séparateur comme `join()`. Voici un exemple :

```
let colors = ["Red", "Green", "Blue"];
document.write(colors.toString()); // Prints: Red,Green,Blue
```

Extraction d'une partie d'un tableau

Si vous souhaitez extraire une partie d'un tableau (c'est-à-dire un sous-tableau) mais conserver le tableau d'origine intact, vous pouvez utiliser la méthode `slice()`. Cette méthode prend 2 paramètres : index de début (index auquel commencer l'extraction) et un index de fin facultatif (index avant lequel terminer l'extraction), comme `arr.slice(startIndex, endIndex)`. Voici un exemple :

```
let fruits = ["Apple", "Banana", "Mango", "Orange", "Papaya"];  
let subarr = fruits.slice(1, 3);  
document.write(subarr); // Prints: Banana,Mango
```

Fusion de tableaux baies ou plus

La méthode `concat()` peut être utilisée pour fusionner ou combiner deux tableaux ou plus. Cette méthode ne modifie pas les tableaux existants, mais renvoie un nouveau tableau. Par exemple:

```
let animals = pets.concat(wilds);  
document.write(animals); // Prints:Chat,Chien,Perroquet,Tigre,Loup,Eléphant
```

Recherche dans un tableau

Si vous souhaitez rechercher une valeur spécifique dans un tableau, vous pouvez simplement utiliser `indexOf()` et `lastIndexOf()`. Si la valeur est trouvée, les deux méthodes renvoient un index représentant l'élément du tableau. Si la valeur n'est pas trouvée, `-1` est retourné. La méthode `indexOf()` renvoie le premier trouvé, tandis que la renvoie le dernier trouvé `lastIndexOf()`.

```
let fruits = ["Apple", "Banana", "Mango", "Orange", "Papaya"];

document.write(fruits.indexOf("Apple") + "<br>"); // Prints: 0
document.write(fruits.indexOf("Banana") + "<br>"); // Prints: 1
document.write(fruits.indexOf("Pineapple")); // Prints: -1
```